Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Prof. Dr.-Ing. Sebastian Schlesinger
Fachbereich 2 - Duales Studium Wirtschaft & Technik

Übung

Embedded Systems Engineering

Embedded Systems Grundlagen

HWR Berlin, Wintersemester 2026

Prof. Dr.-Ing. Sebastian Schlesinger — Übung
Blatt 1

Aufgabe 1 (Embedded Systems - Grundlagen) (3 Punkte)

1. Was sind 'eingebettete' Systeme? Überlegen Sie sich Beispiele und klassifizieren Sie diese nach den in der Vorlesung vorgestellten Kriterien zur Klassifikation eingebetteter Software.
2. Wie lässt sich der Aufbau eingebetteter Systeme beschreiben?
3. Was bezeichnet der Begriff Regelstrecke?

— Lösung Anfang —

Zu 1: Begriff und Klassifikation

Definition. Ein eingebettetes System ist ein informationsverarbeitendes System aus Hard- und Software, das in ein umgebendes technisches System integriert ist und dieses *steuert bzw. regelt*. Charakteristisch sind:

- Es steuert eine (meist analoge) technische Umgebung mit digitalen Komponenten.
- Die Kopplung an die Umgebung erfolgt über **Sensoren** (Messgrößen) und **Aktoren** (Stellgrößen).
- Es ist für die Nutzerin in der Regel nicht als „Computer" sichtbar und nicht frei programmierbar.
- Sein Zweck ist nicht die Berechnung einer mathematischen Funktion, sondern die *dauerhafte Interaktion* mit der Umgebung; es terminiert im Normalbetrieb nicht.
- Es unterliegt typischerweise Ressourcen-, Energie-, Bauraum- und Stückkostenbeschränkungen.

Zur Einordnung: Über 98 % aller weltweit gefertigten Prozessoren stecken in eingebetteten Systemen. Werden eingebettete Systeme vernetzt und mit der Cyber-Welt rückgekoppelt, spricht man von *Cyber-Physical Systems* bzw. *Internet of Things*.

Klassifikationskriterien aus der Vorlesung:

(a) transformatorisch vs. reaktiv

(b) diskretes vs. kontinuierliches Zeitverhalten

(c) verteilte vs. monolithische Systeme

(d) sicherheitskritisch vs. nicht sicherheitskritisch (z. B. Infotainment)

(e) Massenmarkt-Gesetzen unterliegend (Stückkosten-/Time-to-Market-Druck) vs. Kleinserie

Ergänzend die in der Vorlesung gezeigte Schachtelung: *eingebettete Systeme* ⊇ *reaktive Systeme* ⊇ *Echtzeitsysteme*, sowie die Unterscheidung *harte* vs. *weiche* Echtzeit.

Grenzfälle (Diskussion aus der Vorlesung). Smartphone, Smart-TV und Cloud-Dienst sind keine eingebetteten Systeme im engeren Sinn: Sie sind frei programmierbare Allzweckrechner und steuern keine technische Umgebung. Sie *enthalten* allerdings eingebettete Teilsysteme (Baseband-Prozessor, Kamera- und

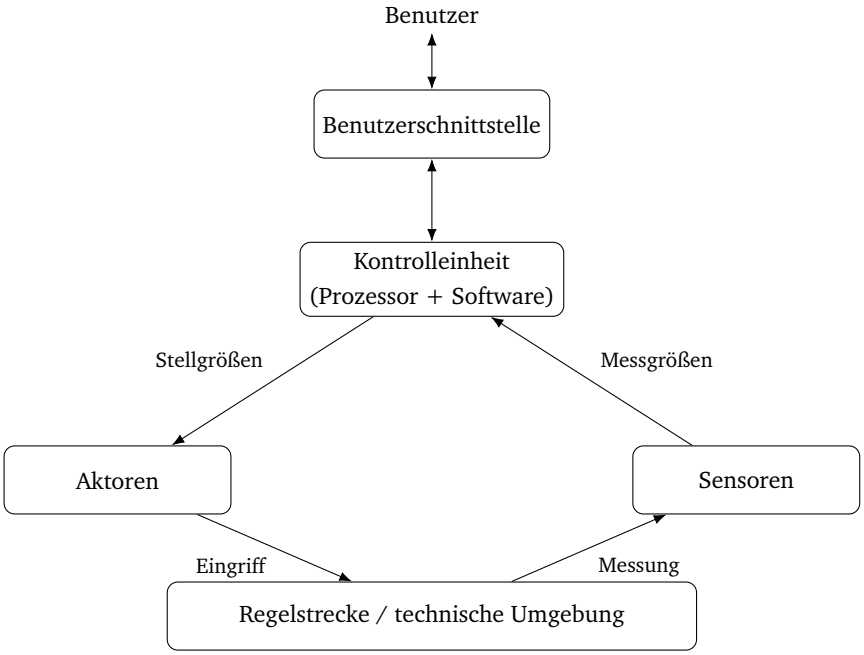
Beispiel	**transf./ reaktiv**	**Zeitverhalten**	**Architektur**	**Kritikalität / Echtzeit**	**Markt**
ABS-/ESP-Steuergerät	reaktiv	hybrid: kontinuierliche Radschlupf-Regelung, diskreter Regelzyklus	verteilt (mehrere Steuergeräte, CAN/FlexRay)	sicherheitskritisch, harte Echtzeit (ISO 26262)	Massenmarkt
Herzschrittmacher	reaktiv	hybrid: kontinuierliche Messung, diskrete Stimulation	monolithisch	höchst sicherheitskritisch, harte Echtzeit	Kleinserie (hoher Zertifizierungsaufwand)
Waschmaschinensteuerung	überwiegend reaktiv; Programmablauf transformatorisch geprägt	diskrete Ablaufsteuerung, kontinuierlich bei Temperatur/Drehzahl	monolithisch	kaum kritisch (Überlauf-/Verbrühschutz), weiche Echtzeit	Massenmarkt
Infotainment-/ Navigationssystem	reaktiv (Benutzerinteraktion)	diskret	verteilt (Fahrzeugbus, Mobilfunk/Cloud)	nicht sicherheitskritisch, weiche Echtzeit; Rückwirkungsfreiheit nötig	Massenmarkt
Industrieroboter	reaktiv	hybrid: kontinuierliche Achsregelung, diskrete Ablauflogik	verteilt (SPS, Feldbus, Antriebe)	sicherheitskritisch (Personenschutz), harte Echtzeit	Kleinserie
Flugregler einer Drohne	reaktiv	kontinuierlich (Lageregelung im kHz-Bereich)	monolithisch, bei Redundanz verteilt	sicherheitskritisch, harte Echtzeit	Massenmarkt (Consumer)
Smart Meter	eher transformatorisch (Erfassen, Aggregieren, Übertragen)	diskret	verteilt	nicht safety-, aber *security*-kritisch; weiche Echtzeit	Massenmarkt

Energiemanagement-Controller). Eine Smart-Home-Anwendung ist ein Grenzfall: Die Feldgeräte (Thermostat, Rollladenaktor) sind eingebettet, die App-/Cloud-Schicht nicht.

Zu 2: Aufbau eingebetteter Systeme

Der Aufbau lässt sich auf drei Ebenen beschreiben:

(a) Funktionale Struktur / Wirkungskette. Die Kontrolleinheit greift über Aktoren in die Umgebung ein und beobachtet sie über Sensoren – es entsteht ein geschlossener Wirkungskreis. Zusätzlich gibt es eine Benutzerschnittstelle zum Menschen.



(b) Hardware-Architektur. Ein oder mehrere Prozessoren mit zugehöriger Software, Speicher, spezielle Hardware (ASIC, FPGA, Signalprozessoren) und Standardkomponenten; die Kopplung zur technischen Umgebung erfolgt über Sensorik und Aktorik, die Kopplung untereinander über Busse und Netzwerke (CAN, FlexRay, Ethernet).

(c) Software-Architektur. Typischerweise geschichtet: Hardware-Abstraktion und Treiber, darüber ein (Echtzeit-)Betriebssystem oder ein reiner Bare-Metal-Scheduler, Kommunikationsstacks sowie die eigentliche Steuerungs-, Regelungs- und Diagnoselogik, realisiert als Tasks und Interrupt-Service-Routinen.

Zu 3: Regelstrecke

Der Begriff stammt aus der Regelungstechnik. Die **Regelstrecke** (engl. *plant*, *controlled system*) ist derjenige Teil der technischen bzw. physikalischen Umgebung, dessen Verhalten geregelt werden soll – also das *zu beeinflussende System* zwischen Stellgröße (Eingang) und Regelgröße (Ausgang). Sie gehört ausdrücklich *nicht* zum Rechner, sondern zum Prozess.

Im Regelkreis unterscheidet man:

- **Führungsgröße** w (Sollwert), **Regelgröße** x (Istwert), **Regeldifferenz** $e = w - x$
- **Stellgröße** u: Ausgang des Reglers, wirkt über den Aktor auf die Strecke
- **Störgröße** z: von außen auf die Strecke wirkende Einflüsse

Abzugrenzen ist die *Regelung* (closed loop, mit Rückführung der Regelgröße) von der *Steuerung* (open loop, ohne Rückführung).

Beispiel Tempomat: Regelstrecke ist die Längsdynamik des Fahrzeugs (Motor, Antriebsstrang, Fahrwiderstände); Stellgröße ist das angeforderte Antriebsmoment, Regelgröße die Geschwindigkeit, Störgrößen sind Steigung und Wind.

Bedeutung für eingebettete Systeme: Die Regelstrecke verhält sich in aller Regel *kontinuierlich* (beschrieben durch Differentialgleichungen), der Regler arbeitet *zeitdiskret* mit fester Abtastzeit – zusammen ergibt sich ein *hybrides* System. Abtastrate, Totzeiten und Jitter gehen unmittelbar in Regelgüte und Stabilität ein. Genau deshalb ist Zeit in eingebetteten Systemen eine Frage der Korrektheit und nicht der Performanz.

— Lösung Ende —

Aufgabe 2 (Embedded Intro - Teil 1) (3 Punkte)

In [1] steht ein Überblicksartikel zum Thema Embedded Software. Dieser gibt einen sehr guten Einblick in die speziellen Probleme, denen die Entwickler eingebetteter Software gegenüberstehen. Lesen Sie die Abschnitte 1 bis 3 und bereiten Sie sich auf folgende Fragen vor:

1. Inwiefern unterscheidet sich Embedded Software von 'klassischer' Software? Welche unterschiedlichen Anforderungen gibt es? Warum ist das für uns als Software-Entwickler wichtig und wie können wir damit umgehen?

2. Diskutieren Sie die Begriffe Timeliness, Concurrency, Liveness, Interfaces, Heterogeneity, Reactivity und deren Bedeutung für die Entwickler eingebetteter Systeme.

3. Welche Schwächen (laut Autor) haben die vorherrschenden Ansötze der Softwaretechnik für Embedded Software (Objektorientierung, Hardware-Beschreibungssprachen und Echtzeitbetriebssysteme)? Teilen Sie die Meinung des Autors?

— Lösung Anfang —

Zu 1: Embedded Software vs. klassische Software

Lees Kernthese lautet: *„Embedded software is not just software on small computers."* Der Unterschied ist kein gradueller (weniger Speicher, langsamerer Prozessor), sondern ein prinzipieller.

Klassisches Bild von Software: Ein Programm implementiert eine berechenbare Funktion Eingabe $\mapsto$ Ausgabe. Terminierung ist erwünscht, die Umgebung wird wegabstrahiert, und Zeit ist eine reine Performanzfrage – schneller ist besser. Die gesamte Abstraktionshierarchie der Informatik (Befehlssatz, Programmiersprache, Betriebssystem) ist genau darauf ausgelegt, physikalische Eigenschaften *auszublenden*.

Embedded Software dagegen:

- ist in einen physikalischen Prozess eingebettet; die Wirkung in der physikalischen Welt *ist* der Zweck und kein Seiteneffekt;
- terminiert nicht – Nichttermination ist Anforderung, im klassischen Berechenbarkeitsmodell dagegen ein Fehlerfall;
- ist *zeitbehaftet*: Korrektheit hängt vom Zeitpunkt ab. Häufig gilt nicht „so schnell wie möglich", sondern „zum richtigen Zeitpunkt" – zu früh kann ebenso falsch sein wie zu spät;
- ist *inhärent nebenläufig*, weil die physikalische Welt nebenläufig ist und nicht sequenziell;
- unterliegt harten Ressourcen-, Energie- und Stückkostenschranken;
- ist heterogen (Hardware/Software, kontinuierlich/diskret, verschiedene Protokolle);
- muss hochzuverlässig sein: Feldupdates sind teuer oder unmöglich, Fehler haben physikalische Konsequenzen.

Verschärfend kommt hinzu, dass moderne Rechnerarchitekturen (Caches, Pipelines, spekulative Ausführung, Sprungvorhersage, dynamische Taktung) den *Durchschnitt* verbessern und die *Vorhersagbarkeit* verschlechtern. Der Entwicklerin wird damit die Kontrolle über das Zeitverhalten genommen – genau über die Eigenschaft, die hier korrektheitsrelevant ist.

Warum ist das für uns wichtig, und wie gehen wir damit um?

- Nicht-funktionale Anforderungen (Zeit, Nebenläufigkeit, Ressourcen, Sicherheit) gehören in die *Spezifikation* und in das *Modell* – sie lassen sich nicht nachträglich „hinein-optimieren".

- Modellbasierte Entwicklung mit Modellen, die eine *formale Semantik* besitzen: endliche Automaten, Statecharts, Timed Automata, signalflussorientierte Modelle (Simulink). Darauf aufbauend Simulation, Model Checking und Codegenerierung.
- Analysierbare Architekturen: statische Schedules, WCET-Analyse, Verzicht auf schwer vorhersagbare Konstrukte (dynamische Speicherallokation, unbeschränkte Rekursion, Garbage Collection), Coding Standards wie MISRA-C, speichersichere Sprachen wie Rust.
- Trennung von Funktionalität und Sicherheitsüberwachung: ein eigener Safety-Monitor bzw. Watchdog neben der eigentlichen Steuerungslogik.

Zu 2: Die Begriffe im Einzelnen

- **Timeliness (Zeitbehaftung, Rechtzeitigkeit).** Die Korrektheit hängt vom Zeitpunkt der Reaktion ab. Klassische Programmiersprachen und ihre Semantiken kennen keinen Zeitbegriff; auch klassische Temporallogik beschreibt nur *qualitative* Zeit („immer“, „irgendwann“), aber keine exakten Schranken. *Für Entwickler:* Deadlines, Perioden und Jitter müssen Teil von Spezifikation *und* Analyse werden – über Timed Automata, zeitbehaftete Logiken (TCTL), WCET- und Schedulability-Analyse.
- **Concurrency (Nebenläufigkeit).** Mehrere unabhängige Kontrollflüsse: Das System reagiert auf eine Menge von Sensoren und steuert „gleichzeitig“ Aktoren an. Die etablierten Lösungen der Betriebssystemwelt (Scheduling, Semaphore, Monitore, Interrupts) funktionieren, sind aber schwer vorhersagbar und fehleranfällig – Race Conditions, Deadlocks, Prioritätsinversion. *Für Entwickler:* nebenläufige Modelle mit klarer Semantik verwenden, Zustandsteilung minimieren (Message Passing statt Shared Memory), kritische Abschnitte verifizieren statt testen.
- **Liveness (Lebendigkeit).** Das System darf weder terminieren noch blockieren; es muss „immer wieder etwas Gutes passieren“. Auszuschließen sind Deadlock (alle Prozesse warten auf ein Ereignis, das nur ein anderer Prozess der Menge auslösen kann), Livelock und Starvation. *Für Entwickler:* Lebendigkeitseigenschaften explizit formulieren (z. B. in LTL als $\square\diamond\,\varphi$) und per Model Checking nachweisen.
- **Interfaces (Schnittstellen).** In klassischer Software beschreibt eine Schnittstelle nur Typen von Parametern und Rückgabewerten und impliziert eine Übergabe des Kontrollflusses. Für eingebettete Systeme müsste sie darüber hinaus die *Dynamik* beschreiben: zulässige Aufrufreihenfolgen und Protokolle, Zeitverhalten, Nebenläufigkeit, Ressourcenverbrauch. *Für Entwickler:* Lee fordert „system-level types“ bzw. Interface-Theorien, mit denen sich die Verträglichkeit einer Komposition ähnlich statisch prüfen lässt wie heute Datentypen.
- **Heterogeneity (Heterogenität).** Gemischt werden Hardware und Software, kontinuierliche und diskrete Anteile, synchrone und asynchrone Abläufe, blockierende und nichtblockierende Kommunikation, kontroll- und datenflussorientierte Sprachen. *Für Entwickler:* Ein einziger Formalismus reicht nicht; erforderlich sind hierarchisch heterogene Modelle, eine sauber definierte Semantik an den Modellgrenzen und Co-Simulation.
- **Reactivity (Reaktivität).** Das System steht in kontinuierlicher Interaktion mit der Umgebung und arbeitet in *deren* Tempo, nicht im eigenen. Ein „verpasstes“ Ereignis kann schwere Folgen haben. *Für Entwickler:* ereignisorientierte Modelle (Automaten, Statecharts), reaktive Sprachen (Esterel, Lustre, Ada) oder Erweiterungen etablierter Sprachen (SystemC, POSIX für C, RTSJ für Java); auf Implementierungsebene Interrupt-Architektur, Pufferung und definiertes Überlastverhalten.

Zu 3: Schwächen der vorherrschenden Ansätze

Objektorientierung.

- Sie abstrahiert genau das weg, worauf es ankommt: Zeit, Nebenläufigkeit und die physikalische Realisierung.

- Schnittstellen sind Methodensignaturen. Sie sagen nichts über Zeitverhalten, Protokoll oder Nebenläufigkeit aus; die Übergabe des Kontrollflusses (*transfer of control*) ist die falsche Abstraktion für nebenläufig interagierende Komponenten.
- Objekte sind passiv – sie müssen aufgerufen werden. Nebenläufigkeit wird nachträglich über Threads und Monitore angeflanscht, ohne Kompositionssemantik: Aus der Korrektheit der Teile folgt nicht die Korrektheit der Komposition.
- Dynamische Bindung, dynamische Allokation und Garbage Collection erschweren die Vorhersage von Zeit- und Speicherverhalten.

Hardware-Beschreibungssprachen (VHDL, Verilog).

- Stärke: Zeit und Nebenläufigkeit sind Teil der Semantik (Discrete-Event-Modell) – genau das, was der Objektorientierung fehlt.
- Schwäche: Das Abstraktionsniveau ist für komplexe Softwarefunktionalität zu niedrig; es fehlen Strukturierungs- und Abstraktionsmittel, wie sie große Softwaresysteme benötigen.
- Die Semantik ist auf Simulation und Synthese von Hardware zugeschnitten und in Details (Delta-Zyklen, Ereignisse mit gleichem Zeitstempel) subtil und wenig intuitiv. SystemC ist der Versuch, dieses Modell in die Softwarewelt (C++) zu übertragen.

Echtzeitbetriebssysteme.

- Zeitanforderungen werden nur *indirekt* über Prioritäten ausgedrückt. Priorität ist eine schwache, globale und nicht-kompositionale Codierung einer Deadline.
- Daraus folgen die klassischen Probleme: Prioritätsinversion (prominentes Beispiel: Mars Pathfinder), Deadlocks, Starvation.
- Es gibt keine Kompositionalität: Das Hinzufügen einer Task kann das Zeitverhalten bestehender Tasks zerstören. Analysen wie RMS oder EDF setzen starke Annahmen voraus (Periodizität, Unabhängigkeit, bekannte WCET), die in realen Systemen selten alle gelten.
- Threads und Locks sind für Entwickler notorisch schwer korrekt zu verwenden; der resultierende Nichtdeterminismus macht Fehler schlecht reproduzierbar und Tests wenig aussagekräftig.
- Grundsätzlich: Ein RTOS löst ein Ressourcen-Multiplexing-Problem, liefert aber kein Modell, in dem Zeit Bestandteil der *Semantik* ist.

Eigene Stellungnahme (Diskussionsaufgabe, es gibt keine eindeutig richtige Antwort). Die Analyse ist mehr als zwanzig Jahre später weitgehend bestätigt: Modellbasierte Entwicklung mit Simulink/Stateflow, SCADE und Statecharts ist in Automotive und Avionik Standard, Timed Automata und Model Checking sind etablierte Werkzeuge, und mit Rust setzt sich eine Sprache durch, die Data-Race-Freiheit zur Übersetzungszeit garantiert. Einschränkend ist festzuhalten: Objektorientierung und Echtzeitbetriebssysteme sind in der Praxis außerordentlich erfolgreich, weil sie Wiederverwendung, Werkzeugunterstützung und verfügbares Personal mitbringen. Lees Kritik trifft sie weniger als *Implementierungstechnik*, sondern als *alleinige Entwurfsabstraktion*. Der pragmatische Schluss lautet daher: auf Entwurfsebene mit Modellen arbeiten, die Zeit und Nebenläufigkeit semantisch erfassen, und die klassischen Techniken darunter als Implementierungsschicht nutzen.

— Lösung Ende —

Aufgabe 3 (Embedded Intro - Teil 2) (3 Punkte)

Lesen Sie nun die verbleibenden Abschnitte des Papers [1] und bereiten Sie sich auf folgende Fragen vor:

1. Welchen Ansatz schlägt der Autor vor? Inwiefern vermeidet er die beschriebenen Schwächen der anderen Ansätze? Was halten Sie von dem Ansatz?

2. Was bedeutet der Begriff Model of Computation und welchem Zweck dient er?

3. Im Artikel werden die Models of Computation der folgenden Konzepte diskutiert. Notieren Sie zu jedem der genannten Konzepte 1-2 Stichpunkte, die bezeichnend für das jeweilige Model of Computation sind.

   a) Datenflussmodelle

   b) Time Triggered Architectures (TTA)

   c) Synchronous / Reactive (SR)

   d) Discrete Event models (DE)

   e) Process Networks

   f) Rendezvous

   g) Publish and Subscribe

   h) Continuous Time models (CT)

   i) Finite State Machines

4. Was sind Vor- und Nachteile der Verwendung allgemeiner oder spezialisierter Modelle zur Modellierung eingebetteter Systeme? Welche Vor- und Nachteile hat die Kombination mehrerer Modelle?

— Lösung Anfang —

Zu 1: Lees Ansatz

Lee schlägt vor, von der *objektorientierten* zur *aktorenorientierten* Entwurfssicht (*actor-oriented design*) überzugehen und die Semantik der Komposition explizit zu machen:

- **Aktoren statt Objekte.** Komponenten sind nebenläufig, besitzen *Ports* und kommunizieren über Datenströme bzw. Signale. Die Schnittstelle ist nicht der Methodenaufruf mit Kontrollflussübergabe, sondern der Datenaustausch.
- **Explizites Model of Computation.** Wie Komponenten nebenläufig ausgeführt werden und wie sie kommunizieren, legt nicht die Komponente fest, sondern das umgebende Berechnungsmodell (in Ptolemy II durch einen *Director* repräsentiert). Die „Gesetze“, nach denen die Komposition arbeitet, sind damit Teil des Modells und nicht implizite Eigenschaft der Implementierung.
- **Hierarchische Heterogenität.** Verschiedene Berechnungsmodelle werden hierarchisch geschachtelt, sodass jedes Teilproblem im dafür passenden Modell beschrieben wird – etwa kontinuierliche Zeit für die Regelstrecke, endliche Automaten für Betriebsmodi, Datenfluss für Signalverarbeitung, Discrete Event für die Buskommunikation.
- **Interface-Theorien / „system-level types“.** Schnittstellen beschreiben auch Dynamik, Zeit und Nebenläufigkeit, sodass sich unverträgliche Kompositionen statisch erkennen lassen – analog zur Typprüfung, aber auf Systemebene.

Inwiefern vermeidet das die beschriebenen Schwächen? Zeit und Nebenläufigkeit sind Bestandteil der Semantik und nicht nachträglich angeflanscht (Gegenentwurf zur Objektorientierung). Das Abstraktionsniveau bleibt softwaretauglich und skaliert auf komplexe Funktionalität (Gegenentwurf zu den Hardware-Beschreibungssprachen). Das Zeitverhalten ergibt sich aus dem Modell statt aus Prioritäten und Scheduler-Heuristiken, und durch bewusst *eingeschränkte*, domänenspezifische Modelle wird Analysierbarkeit überhaupt erst möglich (Gegenentwurf zum RTOS-Ansatz).

Einschätzung (Diskussion). Für den Ansatz spricht, dass er sich in der Praxis durchgesetzt hat, wenn auch nicht unter Lees Namen: Simulink/Stateflow, SCADE, Modelica und AUTOSAR sind im Kern aktorenorientierte, modellbasierte Werkzeugketten mit Codegenerierung; ROS 2 realisiert ein Publish-Subscribe-Berechnungsmodell. Kritisch anzumerken sind die starke Werkzeugabhängigkeit und das Risiko des Vendor-Lock-in, der Einarbeitungsaufwand, die Gefahr auseinanderlaufender Modelle und Implementierungen sowie der schwierigste Punkt des Ansatzes selbst: eine saubere Semantik *an den Grenzen* zwischen unterschiedlichen Berechnungsmodellen. Ptolemy II selbst ist ein Forschungswerkzeug geblieben – die Ideen sind es nicht.

Zu 2: Model of Computation

Ein *Model of Computation* (MoC, Berechnungsmodell) ist die Menge der Regeln, die die *Interaktion von Komponenten* innerhalb eines Modells bestimmen. Lee beschreibt es bildhaft als die „Gesetze der Physik“, denen die Komponenten einer Komposition unterliegen. Ein MoC legt fest:

- Was ist eine Komponente, und wann wird sie aktiv (Aktivierungs-/Feuerregel, Scheduling)?
- Wie wird kommuniziert – blockierend oder nichtblockierend, gepuffert oder synchron, gerichtet oder anonym?
- Welche Rolle spielt Zeit: gar keine, logische Ordnung, diskrete Ticks, Zeitstempel oder kontinuierliche Zeit? Was bedeutet „gleichzeitig“?
- Wann ist eine Komposition wohldefiniert, determiniert und ausführbar?

Zweck:

- **Formale Semantik** als Grundlage für Simulation, Verifikation und Codegenerierung – ohne sie ist ein Modell nur ein Bild.
- **Kompositionalität:** Das Verhalten der Zusammensetzung folgt aus den Regeln und muss nicht für jede Kombination neu ermittelt werden.
- **Analysierbarkeit durch gezielte Einschränkung** der Ausdrucksstärke – etwa statisches Scheduling, beschränkte Puffergrößen oder entscheidbare Deadlock-Freiheit.
- **Passung zur Domäne** und damit eine gemeinsame Sprache zwischen Domänenexpertin und Softwareentwickler.

Zu 3: Charakteristika der einzelnen Berechnungsmodelle

(a) Datenflussmodelle. Aktoren feuern datengetrieben, sobald genügend Token an ihren Eingängen anliegen; ein globaler Zeitbegriff fehlt. Im Spezialfall *Synchronous Dataflow* (SDF) sind die Token-Raten statisch bekannt, wodurch sich Schedule, Puffergrößen und Deadlock-Freiheit bereits zur Übersetzungszeit bestimmen lassen. Natürlicher Anwendungsbereich: Signalverarbeitung und Streaming.

(b) Time Triggered Architectures (TTA). Rechnen und Kommunizieren folgen einem *statischen, global zeitgesteuerten Fahrplan* (TDMA) auf Basis einer synchronisierten globalen Zeit; nichts geschieht ereignisgetrieben. Das ergibt maximale Vorhersagbarkeit, Testbarkeit und Fehlertoleranz, erkauft mit Unflexibilität gegenüber sporadischen Ereignissen und einer Auslegung sämtlicher Ressourcen auf den Worst Case.

(c) Synchronous / Reactive (SR). Es gilt die *Synchronie-Hypothese*: Reaktionen benötigen keine Zeit, das System durchläuft diskrete Ticks, in denen alle Komponenten gleichzeitig auf die anliegenden Eingaben reagieren. Die Semantik ist ein Fixpunkt, weshalb eine Kausalitätsanalyse nötig ist (Ausschluss von Kausalitätsschleifen). Determiniert und sehr gut analysierbar (Esterel, Lustre, SCADE), aber global eng gekoppelt und damit auf verteilten Plattformen teuer.

(d) Discrete Event models (DE). Ereignisse tragen *Zeitstempel*; die Ausführung verarbeitet sie in chronologischer Reihenfolge aus einer globalen Ereigniswarteschlange. Zeit ist explizit Teil der Semantik. Grundlage von Hardware- und Netzwerksimulatoren (VHDL, Verilog, SystemC). Kritische Punkte: gleichzeitige Ereignisse und Null-Verzögerungs-Rückkopplungen (Delta-Zyklen, „superdense time“) sowie die teure globale Zeitordnung in verteilten Systemen.

(e) Process Networks (Kahn Process Networks). Nebenläufige Prozesse kommunizieren über konzeptuell unbeschränkte FIFO-Kanäle; Lesen blockiert, Schreiben blockiert nie. Das Ergebnis ist *determiniert und unabhängig von der Ausführungsreihenfolge* – eine sehr starke Eigenschaft. Ein Zeitbegriff fehlt; Beschränktheit der Puffer und Deadlock-Freiheit sind im Allgemeinen unentscheidbar.

(f) Rendezvous. Synchrone, ungepufferte Punkt-zu-Punkt-Kommunikation: Sender und Empfänger warten aufeinander, der Datenaustausch findet atomar im „Treffpunkt“ statt (CSP, CCS, Occam, Ada). Gut geeignet zur Modellierung geteilter Ressourcen und formal hervorragend analysierbar (Prozessalgebren, Bisimulation); dafür starke Kopplung und Deadlock-Gefahr, Nichtdeterminismus über bewachte Auswahl.

(g) Publish and Subscribe. Anonyme, lose Kopplung über benannte Ereignisströme (Topics); Produzenten kennen ihre Konsumenten nicht. Sehr gut für dynamische, verteilte und skalierbare Systeme (heute DDS/ROS 2, MQTT). Der Preis ist eine schwach definierte Semantik bezüglich Reihenfolge, Zeit und Zustellgarantien – Determinismus und harte Echtzeit sind entsprechend schwer zuzusichern.

(h) Continuous Time models (CT). Das Verhalten wird durch Differentialgleichungen über einem kontinuierlichen Zeitbereich beschrieben; ausgeführt wird es näherungsweise durch einen numerischen Solver (Schrittweitensteuerung, Nullstellenerkennung). Es ist das passende Modell für die physikalische Regelstrecke; in Kombination mit diskreten Anteilen entstehen *hybride* Systeme (Simulink, Modelica).

(i) Finite State Machines. Endliche Zustandsmenge mit Transitionen; das Verhalten ist vollständig aufzählbar und damit sehr gut verifizierbar (Erreichbarkeitsanalyse, Model Checking). Für sich genommen kennen FSM weder Nebenläufigkeit noch Zeit, und bei Komposition droht Zustandsexplosion – daher die Erweiterungen: Statecharts (Hierarchie und Nebenläufigkeit), Timed Automata (Zeit) und hybride Automaten (FSM kombiniert mit CT).

Zu 4: Allgemeine vs. spezialisierte Modelle

Allgemeine Modelle (Turing-mächtige Sprachen, UML, sehr ausdrucksstarke Berechnungsmodelle):

- *Vorteile:* Alles ist modellierbar, ein Formalismus deckt das Gesamtsystem ab, keine Modellierungs-Verrenkungen nötig; breite Werkzeug- und Personalbasis.
- *Nachteile:* Analyseaussagen werden schwach oder unentscheidbar (Halteproblem, Erreichbarkeit); statische Garantien zu Zeit und Speicher sind nicht zu bekommen; die Semantik ist häufig unterspezifiziert – bei UML bedeutet dasselbe Diagramm in verschiedenen Werkzeugen Verschiedenes. Zudem sind solche Modelle selten domänennah.

Spezialisierte Modelle (SDF, SR, Timed Automata, TTA, CT):

- *Vorteile:* Die eingeschränkte Ausdrucksstärke *kauft* Analysierbarkeit – statisches Scheduling, beschränkter Speicherbedarf, Determinismus, Deadlock-Freiheit, Model Checking, WCET-Analyse und verlässliche Codegenerierung. Die Notation ist nah an der Domäne und damit für Domänenexperten lesbar.
- *Nachteile:* Nicht alles lässt sich ausdrücken. Erzwungene Workarounds zerstören häufig genau die Analysevorteile, für die man das Modell gewählt hat. Hinzu kommen Lernaufwand, Werkzeugvielfalt und Lock-in-Risiken; die Grenzen des Modells müssen verstanden sein.

Kombination mehrerer Modelle (hierarchische Heterogenität):

- *Vorteile:* Jedes Teilproblem wird im passenden Modell beschrieben und behält dort seine maximale Analysierbarkeit. Das entspricht der tatsächlich heterogenen Natur eingebetteter Systeme – kontinuierliche Regelstrecke, diskrete Steuerung, ereignisbasierte Kommunikation. Lokale Verifikation und Wiederverwendung von Komponenten werden dadurch erst praktikabel.
- *Nachteile:* Die Semantik *an den Modellgrenzen* muss präzise definiert sein – etwa die Abtastung beim Übergang CT → FSM oder die Zeitstempel-Interpretation beim Übergang DE → SR. Genau dort entstehen erfahrungsgemäß die Fehler. Globale Eigenschaften wie Ende-zu-Ende-Deadlines lassen sich über Modellgrenzen hinweg deutlich schwerer beweisen als innerhalb eines Modells. Hinzu kommen konzeptionelle Komplexität, Integration der Werkzeugkette, Co-Simulation und Schulungsaufwand.

Lees Antwort lautet entsprechend: Die Kombination ist *notwendig*, aber nur mit wohldefinierten Schnittstellen zwischen den Berechnungsmodellen. In dieser Vorlesung sehen wir genau dieses Muster am Robotikbeispiel: Statecharts für die Betriebsmodi, Timed Automata für Deadlines und Watchdogs, Simulink für den Regelkreis und ROS 2 als Publish-Subscribe-Schicht für die Kommunikation.

— Lösung Ende —

Literatur

[1] Edward A. Lee. Embedded software. In: *Advances in Computers*, Vol. 56, pp. 55–95. Elsevier, 2002. DOI: https://doi.org/10.1016/S0065-2458(02)80004-3